

Can Scala 3 Have Macro Annotations Again?

Rebuilding Macro Paradise



Dmytro Mitin

Scala metaprogramming, compilers, and type systems

This presentation

Repository

github.com/DmytroMitin/macroparadise-talk-09-2026

This deck follows `draft/draft_v8.md` from beginning to end. The draft remains the talk's content authority.

Publication after the talk

- QR code remains an open presentation choice.
- SlideShare or another public mirror remains undecided.
- The repository is the stable source.

The publication questions are preserved from the draft as questions, not presented as completed distribution channels.

About myself: mathematics and engineering

Mathematics

- Former university teacher in mathematics
- Analysis, function theory, approximation theory
- Fractal approximation of functions
- Mathematics competitions: training, team leadership, coordination, organization, jury chair

Teaching

- Online courses on dependent types, 2017–2019
- Trainings on Shapeless and macros, 2020

Software engineering

Telecom, fintech, cybersecurity, compilers, instant messengers, document-portal engines, and web programming.

Research interests

Functional programming in Scala and Haskell, metaprogramming, compiler design, type systems, formal verification, and AI systems such as harnesses and coding agents.

Stack Overflow

Top `shapeless` Answerers

Last 30 Days

All Time

1.4k	172		Travis Brown 139k ● 12 ● 385 ● 689
825	77		Miles Sabin 23k ● 6 ● 64 ● 95
375	191		Dmytro Mitin 53.1k ● 3 ● 35 ● 76
72	22		Jasper-M 15.2k ● 2 ● 29 ● 40
66	5		Rex Kerr 168k ● 27 ● 326 ● 411

- Top 15 in Scala by score
- 3rd in Scala by number of answers
- Roughly 1.5% of all Stack Overflow questions tagged `scala` have an answer from me
- 1st in Cats, `scala-reflect`, Scalameta, and Scala 3
- 2nd in Scala macros
- 3rd in Shapeless, after Travis Brown and Miles Sabin

stackoverflow.com/users/5249621/dmytro-mitin

Open-source compiler work

Scala 3

Merged compiler fix for deferred-inline selection after
`summonInline` :

github.com/scala/scala3/pull/26002

Apache Spark

An unmerged Spark pull request:

github.com/apache/spark/pull/38740

The perspective behind this talk

The work combines:

- language-user experience from answering real Scala questions;
- mathematical interest in exact representations;
- practical compiler and metaprogramming experiments;
- a preference for bounded claims backed by executable evidence.

The talk is about a working compiler experiment and the design boundaries revealed by building it.

Scala Semantic Harness for coding agents

```
1  cs install --default-channels=false \  
2    --channel https://raw.githubusercontent.com/\   
3    DmytroMitin/scala-semantic-harness/main/\   
4    distribution/coursier/channel.json \  
5    semantic-scala semantic-scala-mcp
```

Start here

- `skills/semantic-scala/SKILL.md`
- `docs/agent-onboarding.md`
- `docs/early-feedback.md`

Early-feedback guide:

github.com/DmytroMitin/scala-semantic-harness/blob/main/docs/early-feedback.md

github.com/DmytroMitin/scala-semantic-harness

Scala-specific semantic evidence for coding agents, with provenance and explicit limits.

Experimental projects and exact versions

Project	Released line	Current development line
Macro-Paradise	0.1.1	0.2.0-SNAPSHOT
Quasiquotes	0.3.0	compiler-sensitive development continues
AUXify-scala3	0.1.0	0.2.0-SNAPSHOT
Allow Experimental	0.1.0	evolving research project

Qualified Scala lines: 3.3.8 , 3.8.4 , 3.9.0

Released 0.1.1 examples and current-main 0.2.0-SNAPSHOT examples are labelled separately.

Compiler-sensitive APIs, supported source shapes, and compatibility policy continue to evolve.

The question of the talk

Source

Conceptual transformation

```
1  @addFoo
2  class A
```

```
1  class A
2
3  object A:
4    def foo(x: Int): String =
5      x.toString
```

Should ordinary code in the same compilation be allowed to write `A.foo(10)` and have normal name resolution and typing see the generated method?

Generated code and generated API are different requirements

Existing Scala 3 `MacroAnnotation`

- Can transform definitions
- Can create definitions
- Generated definitions are available to the expansion itself
- External visibility is deliberately limited

Paradise-style requirement

The annotation changes the program before ordinary typing.

Generated members, companions, and siblings become part of the API against which later ordinary source is resolved and typed.

```
1  @addFoo class A
2  A.foo(10)
```

The question is not whether Scala 3 can generate code. The question is when the generated definitions enter the ordinary compiler pipeline.

Three execution boundaries

1. Macro library compilation

Compile the macro or handler implementation into executable compiler-side code.

2. Client compilation

Macro implementation code executes here. It produces or transforms compiler trees.

3. Program runtime

The generated ordinary code executes after compilation, with the rest of the application.

Macro implementation execution time equals client compilation time. Generated program runtime remains ordinary runtime.

Ordinary methods and inline methods

Ordinary method

```
1 def add(left: Int, right: Int): Int =  
2   left + right
```

The body executes when the program runs.

Scala 2 @inline

```
1 @inline  
2 def add(left: Int, right: Int): Int =  
3   left + right
```

An optimizer recommendation or hint.

Scala 3 inline

```
1 inline def add(  
2   left: Int,  
3   right: Int  
4 ): Int = left + right
```

Scala 3 gives `inline` language-level compile-time expansion semantics.

The resulting ordinary operations normally execute later at runtime.

Compile-time implementation code produces program code

Scala 2

Scala 3

```
1  def add(left: Int, right: Int): Int =  
2    macro addImpl  
3  
4  def addImpl(c: blackbox.Context)(  
5    left: c.Tree,  
6    right: c.Tree  
7  ): c.Tree = {  
8    import c.universe._  
9    q"$left + $right"  
10 }
```

```
1  inline def add(  
2    left: Int,  
3    right: Int  
4  ): Int = ${ addImpl('left, 'right) }  
5  
6  def addImpl(  
7    left: Expr[Int],  
8    right: Expr[Int]  
9  )(using Quotes): Expr[Int] =  
10    '{ $left + $right }
```

``addImpl`` runs while the caller is compiled. The expression it produces normally runs later.

Macro annotations transform definitions

```
1 @addFoo  
2 class A
```

```
1 class A  
2  
3 object A:  
4   def foo(x: Int): String =  
5     x.toString
```

A macro annotation may transform the annotated definition and generate related definitions.

The strongest use case in this talk requires the generated result to be visible to the ordinary compiler pipeline: ``A.foo(10)``.

Macro-annotation use cases

- Code generation
- Derivation
- Instrumenting code
- Validation and compile-time checks
- Boilerplate members in a class or companion
- Type-class helpers and syntax
- Transforming existing definitions
- Generating sibling definitions

Some uses need only local expansion details. Others require a genuinely source-visible generated API.

From constructors to structural templates

Manual AST

Parse text

Quasiquote

```
1  Apply(  
2    Select(left, "+"),  
3    List(right)  
4  )
```

```
1  c.parse(  
2    s"$left + $right"  
3  )
```

```
1  q"$left + $right"
```

Exact, but verbose and representation-specific.

Produces a tree, but `left` and `right` travel through strings.

Produces a tree while the holes remain structural trees.

Quasiquotes construct and match tree structure

Construction

```
1  source-like template
2    + structural holes
3    + repeated tree holes
4      ↓
5      tree
```

Pattern matching

```
1  tree
2    + source-like pattern
3      ↓
4  captured subtrees
```

Repeated holes can transport sequences of trees.

The holes represent AST pieces, not only interpolated characters.

Parsing sits between manual constructors and quasiquotes, but moving between strings and trees remains a separate problem.

Scala 2

Def macros, tree quasiquotes, and analyzer-integrated Macro
Paradise

Scala 2 reify and splice

```
1  def add(left: Int, right: Int): Int =  
2    macro addImpl  
3  
4  def addImpl(c: blackbox.Context)(  
5    left: c.Expr[Int],  
6    right: c.Expr[Int]  
7  ): c.Expr[Int] = {  
8    import c.universe._  
9    reify {  
10      left.splice + right.splice  
11    }  
12  }
```

The quoted expression must typecheck when the macro implementation is compiled.

That gives typed safety, but it restricts templates that are not valid ordinary Scala expressions in the implementation context.

Where `reify` stops being enough

```
1  def make[A](args: Any*): A = macro makeImpl[A]
2
3  def makeImpl[A: c.WeakTypeTag](c: blackbox.Context)(
4    args: c.Expr[Any]*
5  ): c.Expr[A] = {
6    import c.universe._
7
8    def exprsToExpr[T: WeakTypeTag](
9      exprs: Seq[Expr[T]]
10    ): Expr[Seq[T]] =
11      exprs.foldRight(reify { Seq.empty[T] }) {
12        (expr, acc) => reify {
13          expr.splice ++: acc.splice
14        }
15      }
16
17    reify {
18      new A(exprsToExpr(args).splice)
19      // error: class type required but A found
20    }
21  }
```

The limitation

`A` is available as a type representation, but `new A(...)` cannot appear here as an already typechecked ordinary expression in the required form.

The next step

Construct compiler trees directly with source-like tree quasiquotes instead of asking the template itself to typecheck first.

Scala 2 tree quasiquotes

Expression construction

```
1 def addImpl(c: blackbox.Context)(  
2   left: c.Tree,  
3   right: c.Tree  
4 ): c.Tree = {  
5   import c.universe._  
6   q"$left + $right"  
7 }
```

Constructor plus sequence splice

```
1 def makeImpl[A: c.WeakTypeTag](  
2   c: blackbox.Context  
3 )(args: c.Tree*): c.Tree = {  
4   import c.universe._  
5   q"new ${weakTypeOf[A]}(..$args)"  
6 }
```

`..$args` inserts a sequence of trees structurally.

The quasiquote operates at the compiler-tree level. The template need not be an ordinary expression that already typechecks in the macro implementation.

Scala 2 Macro Paradise: user-facing phase relationship

```
parser
  ↓
macroparadise
  ↓
namer
  ↓
packageobjects
  ↓
typer
  ↓
later phases
```

Generated definitions participated in the normal downstream compiler pipeline.

This is the classic or older conceptual phase relationship. It must not be read as proof that every Paradise operation lived in one conventional extra phase.

The implementation relied on analyzer and macro hooks inside naming and typing.

Scala 2 Paradise used analyzer-plugin machinery

The outer entry point was an ordinary
`scala.tools.nsc.plugins.Plugin`, but:

Representative hooks

```
1  components = Nil
2
3  analyzer.addAnalyzerPlugin(
4    AnalyzerPlugin
5  )
6  analyzer.addMacroPlugin(
7    MacroPlugin
8  )
```

```
1  pluginsEnterStats
2  pluginsEnterSym
3  pluginsEnsureCompanionObject
4  pluginsTypedMacroBody
5  pluginsTypeSig
```

They cooperated with symbol entering, companion handling, and typing macro implementation bodies.

“Analyzer-plugin-based machinery” describes the implementation accurately even though the JAR itself entered as an ordinary Scala 2 compiler plugin.

Scala 2 `macroTransform`: annotation binding

```
1  import scala.annotation.{
2    StaticAnnotation,
3    compileTimeOnly
4  }
5  import scala.language.experimental.macros
6  import scala.reflect.macros.whitebox
7
8  @compileTimeOnly(
9    "enable macro paradise to expand macro annotations"
10 )
11 class addFoo extends StaticAnnotation {
12   def macroTransform(annottees: Any*): Any =
13     macro AddFooMacro.macroTransformImpl
14 }
```

The marker annotation exposes `macroTransform` as a Scala macro implementation.

The analyzer has to understand that binding while it is entering and typing the annotated program.

The generated companion must then join the same downstream naming and typing process.

Scala 2 macroTransform: expansion

```
1  object AddFooMacro {
2    def macroTransformImpl(c: whitebox.Context)(
3      annotees: c.Tree*
4    ): c.Tree = {
5      import c.universe._
6      annotees match {
7        case (cls @ q"$mods class $tpname[..$tparams] $ctorMods(...$paramss)
8              extends { ..$earlydefns } with ..$parents
9              { $self => ..$stats }") :: Nil =>
10          q"""
11            $cls
12            object ${tpname.toTermName} {
13              def foo(x: Int): String = x.toString
14            }
15            """
16      }
17    }
18  }
```

The significant property is phase placement: ordinary naming and typing see the transformed class and generated object.

Scala 3

Typed quotations, reflection, native macro annotations, and the pre-typer alternative

Scala 3 quotations and splices

```
1  inline def add(  
2    left: Int,  
3    right: Int  
4  ): Int = ${ addImpl('left, 'right) }  
5  
6  def addImpl(  
7    left: Expr[Int],  
8    right: Expr[Int]  
9  )(using Quotes): Expr[Int] =  
10    '{ $left + $right }
```

The quotation is typed. That is a major safety improvement.

It is also why Scala 3 quotations do not directly replace arbitrary Scala 2 compiler-tree quasiquotes.

```
1  '{ ... }  term quotation  
2  '[ ... ]  type quotation  
3  $         splice inside quotation  
4  ${ ... }  invoke macro implementation
```

The constructor shape `new A(...)` still needs another representation-level construction strategy when `A` is available only through `Type[A]`.

Scala 3 quoted reflection: constructor application

```
1 inline def make[A](args: Any*): A =  
2   ${ makeImpl[A]('args) }  
3  
4 def makeImpl[A: Type](  
5   args: Expr[Seq[Any]]  
6 )(using Quotes): Expr[A] =  
7   import quotes.reflect.*  
8   args.asTerm.underlying  
9     .asExprOf[Seq[Any]] match  
10     case Varargs(params) =>  
11       Select.overloaded(  
12         New(TypeTree.of[A]),  
13         "<init>",  
14         Nil,  
15         params.map(_.asTerm).toList  
16       ).asExprOf[A]
```

Scala 3 has powerful typed reflection.

The narrower missing convenience is a Scala-2-style `q"..."` or `tq"..."` layer over arbitrary reflection/compiler tree construction and matching.

Manual reflection remains precise, but the author writes the representation structure explicitly.

Explicit reflection construction: discover the constructor shape

```
1 inline def make[A](args: Any*): A =  
2   ${ makeImpl[A]('args) }  
3  
4 def makeImpl[A: Type](  
5   args: Expr[Seq[Any]]  
6 )(using Quotes): Expr[A] =  
7   import quotes.reflect.*  
8  
9   val fields: List[Symbol] =  
10     TypeRepr.of[A].typeSymbol.caseFields  
11  
12   val fieldTypeTrees: List[TypeTree] =  
13     fields.map(  
14       _.tree.asInstanceOf[ValDef].tpt  
15     )
```

The macro obtains the case fields from `TypeRepr.of[A]` and extracts their type trees.

Those trees will guide per-argument casts before the constructor call.

This is typed quoted reflection, not raw pre typer construction.

Explicit reflection construction: build the call

```
1  Apply(  
2    Select.unique(  
3      New(TypeTree.of[A]),  
4      "<init>"  
5    ),  
6    fieldTypeTrees.zipWithIndex.map {  
7      (fieldType, i) =>  
8        TypeApply(  
9          Select.unique(  
10             Apply(  
11               Select.unique(args.asTerm, "apply"),  
12               List(Literal(IntConstant(i)))  
13             ),  
14             "asInstanceOf"  
15           ),  
16           List(fieldType)  
17         )  
18       }  
19   ).asExprOf[A]
```

```
arguments  
  ↓ index each element  
  ↓ cast to the corresponding field type  
  ↓ select constructor  
  ↓ apply constructor  
typed Expr[A]
```

The code is exact, but it exposes the full reflection construction grammar that source-like quasiquotes normally compress.

Native Scala 3 MacroAnnotation

What it can do

- Run in the typed compiler world
- Transform the annotated definition
- Transform its companion
- Add definitions
- Support validation and generated implementation details

The boundary relevant here

New definitions are not visible to ordinary user-written code outside the macro expansion.

That boundary is deliberate. It makes the API unsuitable only for the specific Paradise-style requirement, not for macro annotations in general.

Native MacroAnnotation: typed transformation entry point

```
1  import scala.annotation.{
2    MacroAnnotation,
3    experimental
4  }
5  import scala.quoted.{Quotes, Type, quotes}
6
7  @experimental
8  class addFoo extends MacroAnnotation:
9    def transform(using Quotes)(
10      tree: quotes.reflect.Definition,
11      companion:
12        Option[quotes.reflect.Definition]
13    ): List[quotes.reflect.Definition] =
14      import quotes.reflect.*
15      tree match
16        case ClassDef(className, _, _, _, _) =>
17          // construct typed definitions
18        case _ =>
19          report.errorAndAbort(
20            "@addFoo can annotate only classes"
21          )
```

The input is already in the typed reflection world.

The expansion receives the annotated definition and optional companion, then returns typed definitions.

The following slides retain the draft's full module-construction shape rather than reducing it to a schematic.

Native MacroAnnotation: create the companion symbols

```
1  val modParents = List(TypeTree.of[Object])
2
3  val tpe = MethodType(List("x"))(
4    _ => List(TypeRepr.of[Int]),
5    _ => TypeRepr.of[String]
6  )
7
8  def decls(cls: Symbol): List[Symbol] =
9    List(
10     Symbol.newMethod(
11       cls,
12       "foo",
13       tpe,
14       Flags.EmptyFlags,
15       Symbol.noSymbol
16     )
17   )
```

```
1
2  val mod = Symbol.newModule(
3    Symbol.spliceOwner,
4    className,
5    Flags.EmptyFlags,
6    Flags.EmptyFlags,
7    _ => modParents.map(_.tpe),
8    decls,
9    Symbol.noSymbol
10  )
```

The macro creates a module and method symbol under the current splice owner. This is genuine typed generation; the limitation concerns how far the generated definitions become visible.

Native MacroAnnotation: materialize the definitions

```
1  val cls = mod.moduleClass
2  val fooSym = cls.declaredMethod("foo").head
3
4  val fooDef = DefDef(
5    fooSym,
6    argss => Some(
7      Select.unique(
8        argss.head.head.asInstanceOf[Term],
9        "toString"
10      )
11    )
12  )
13
14  val (modValDef, modClsDef) =
15    ClassDef.module(
16      mod,
17      modParents,
18      body = List(fooDef)
19    )
20
21  val result = List(tree, modValDef, modClsDef)
22  println(result.map(_.show))
23  result
```

The API can create the module value, module class, and `foo` method definition.

The generated definitions remain part of the typed macro-expansion result with deliberately limited visibility to unrelated ordinary user source.

Two macro-annotation contracts

Scala 3 `MacroAnnotation`

Paradise-style transformation

```
typed definition
  ↓ typed macro expansion
generated definitions
  ↓ deliberately limited visibility
expansion result
```

```
parsed untyped program
  ↓ pre-typer rewrite
generated definitions
  ↓ ordinary name resolution + typing
source-visible API
```

Strong typed reflection and an explicit visibility boundary.

Rawer inputs, greater responsibility, and different semantics.

The contrast is typed expansion versus pre-typer program transformation, not “macro annotations versus none.”

Scala 3 compiler-plugin taxonomy

API	Capability	Intended use
<code>StandardPlugin</code>	Contributes one or more <code>PluginPhase</code> s	Normal Scala 3 compiler-plugin mechanism
<code>ResearchPlugin</code>	Receives the whole phase pipeline and may add, remove, replace, or rearrange phases	Experiments with the compiler pipeline itself; documented for nightly or snapshot compilers

Scala 3 has no analyzer-plugin category corresponding directly to Scala 2 analyzer plugins.

“Research project” and `ResearchPlugin` are different descriptions. Macro-Paradise is a research project implemented with `StandardPlugin` .

Why `StandardPlugin` is enough

```
parse ordinary Scala source
  ↓
rewrite parsed untpd program
  ↓
hand final program to ordinary typer
```

Macro-Paradise does not need to:

- replace the parser;
- replace the typer;
- change typing rules;
- rearrange the whole compiler pipeline.

A contributed phase with `runsAfter` and `runsBefore` constraints supplies the required insertion point.

The unusual decision is where the standard plugin phase runs.

The smaller mechanism preserves the ordinary compiler

What remains authoritative

- Dotty's parser semantics
- Dotty's typer semantics
- Normal symbol entering
- Normal overload and reference resolution
- Ordinary downstream typed phases

Practical boundary

- Narrower integration surface
- Stable exact compiler releases can be qualified
- Current lines: 3.3.8 , 3.8.4 , 3.9.0
- No unnecessary whole-pipeline power
- No nightly-only `ResearchPlugin` dependency

The plugin transforms raw syntax once. The stock typer then performs semantic work once on the final program.

Different expansion protocols

Scala 2 Paradise

```
macro binding + analyzer lifecycle
↓
expand while stats and symbols enter
↓
cooperate with namer and typer hooks
```

The implementation extends analyzer operations while their semantic state is developing.

Current Scala 3 Macro-Paradise

```
precompiled handler registry
↓
bounded syntactic untpd transformation
↓
ordinary typer starts on final program
```

The transformation completes before semantic state becomes authoritative.

Why Scala 2 Paradise entered the analyzer lifecycle

- Annotation and macro implementation were connected through compiler symbols and macro bindings.
- Expansion happened while the namer entered stats.
- Generated definitions had to enter the same scope during that process.
- Companion creation and lookup had to cooperate with namer machinery.
- `macroTransform` was a Scala macro implementation.
- Its body and signature needed macro-typer support.

```
pluginsEnterStats / pluginsEnterSym / pluginsEnsureCompanionObject  
+ pluginsTypedMacroBody / pluginsTypeSig
```

The Scala 3 protocol moves prerequisites before typer

```
1  precompiled marker class
2    + runtime @expander metadata
3    + syntactic annotation identity
4    + bounded explicit-import resolver
5    + explicit handler classpath
```

```
pre-typer ExpansionHandler invocation
↓
validated untpd replacement or edit
↓
ordinary Dotty typer enters symbols,
resolves names, and types the result
```

The consumer does not need to type a source `macroTransform` method before the handler can execute.

Before typer: identity is bounded and handlers are executable

Annotation identity

- Qualified names come from raw syntax.
- A small resolver covers the supported explicit-import slice.
- The model does not claim general Scala name resolution.
- Compiled marker metadata names the handler.

Handler implementation

- Already compiled
- Already loadable
- Located on an explicit tool classpath
- Implements a separate handler contract
- Does not require the current consumer compilation to type its body

Raw syntax first, semantic work afterward

Generated symbols

Handlers create or rewrite untyped syntax. They do not mint the final typed symbols.

Stock Dotty receives the transformed program, then enters symbols, resolves references, selects overloads, and types definitions normally.

Companions

Macro-Paradise discovers current primary and companion relationships structurally.

Handlers can request creation, replacement, merging, or deletion. The plugin recomputes final relationships before the ordinary typer begins.

Replacing the raw program before symbol tables become authoritative avoids mutating partially established semantic state.

The price of staying outside typer

- Annotation identity resolution is bounded.
- Handlers receive raw `untpd` and syntactic views.
- General typed symbols and types are unavailable.
- Broad/default handlers must be precompiled.
- General same-file source handlers are unsupported.
- Targets are a bounded package-level Class/Trait/Object slice.
- Type-directed transformations need another design or a later phase.
- Same-module handlers reintroduce explicit staging complexity.

This is a protocol trade, not a theorem that Scala 2 could never have supported a different purely syntactic pre-namer transformer.

Relaxing “precompiled” brings staging back

```
handler source in same module
  ↓ identify selected source
  ↓ suspend dependent consumers
  ↓ compile handler
  ↓ load handler output
  ↓ resume pre-typer expansion
```

Protocol summary

Scala 2 Paradise: integrate expansion into the analyzer while symbols enter.

Current Scala 3 Macro-Paradise: perform a bounded syntactic transformation before typer, then let the ordinary typer process the transformed program once.

Where Macro-Paradise runs

```
source
  ↓
parser
  ↓
Macro-Paradise pre-typer phase
  ↓
typer
  ↓
later compiler phases
```

Pre-typer responsibilities

- Annotation discovery and target admission
- Handler expansion
- Output validation
- Transactional composition
- Package-stat replacement

The plugin entry point extends `StandardPlugin`. The work lives in a `PluginPhase` scheduled after parser and before typer.

A standard plugin in an unusual position

Exact taxonomy

- Extends Scala 3 `StandardPlugin`
- Supplies a `PluginPhase`
- `runsAfter = Set("parser")`
- `runsBefore = Set("typer")`
- Qualifies stable exact Scala releases

Unusual effect

The phase rewrites untyped compilation-unit trees before ordinary typing.

That placement gives source-transformation power one might otherwise associate conceptually with deeper analyzer or research-level extension.

It still does not make the plugin a Scala 3 `ResearchPlugin`.

Generated members, companions, and siblings exist before the typer sees user references such as ``A.foo(10)``.

Scala 3 typed quasiquotes and sequence transport

One ordinary term cannot stand for a sequence

Use the ranked transport

```
1 def makeImpl[A: Type](  
2   args: Expr[Seq[Any]]  
3 )(using Quotes): Expr[A] =  
4   import quotes.reflect.*  
5   qr"new ${TypeRepr.of[A]}(  
6     ..${args.asTerm}  
7   )" .asExprOf[A]
```

This does not work as a sequence splice.

```
1 args.asTerm.underlying  
2 .asExprOf[Seq[Any]] match  
3   case Varargs(params) =>  
4     qr"new ${TypeRepr.of[A]}(  
5       ..${termSplice(  
6         params.map(_.asTerm)  
7       )}  
8     )" .asExprOf[A]
```

The splice explicitly carries a sequence of terms.

Walk by Scala tree representations

Level 0 ordinary values / macro call site

Level 1	Level 2	Level 3	Level 4
Expr	-> (A) ExprImpl	-> tpd.Tree	-> untpd.Tree
	-> (B) q.reflect.Term	->	

The arrows show our walk from a public macro value toward compiler internals.

They do not show compiler phase order, subtyping, or `tpd` compiling into `untpd`.

Actual compiler direction is a separate diagram

```
source
  ↓
parser
  ↓
untpd.Tree
  ↓ typer
tpd.Tree
  ↓
later typed compiler phases
```

Two questions

The **walk map** asks how far down through representations our implementation travels.

The **phase map** asks in what order the compiler processes source.

The relationship between `untpd.Tree` and `tpd.Tree` points in opposite visual directions because the diagrams answer different questions.

Level 0 and Level 1

Level 0: call site

Level 1: Expr

```
1 inline def add(  
2   left: Int,  
3   right: Int  
4 ): Int = ${ addExpr('left, 'right) }
```

```
1 def addExpr(  
2   left: Expr[Int],  
3   right: Expr[Int]  
4 )(using Quotes): Expr[Int] =  
5   '{ $left + $right }
```

Ordinary values enter a staged macro boundary.

Stay in the standard Scala 3 quotation world.

From Expr, two Level 2 branches

Branch A: internal ExprImpl

Branch B: public reflection

```
1  given Contexts.Context =  
2    quotes.asInstanceOf[impl.QuotesImpl].ctx  
3  
4  addExprImpl(  
5    left.asInstanceOf[impl.ExprImpl],  
6    right.asInstanceOf[impl.ExprImpl]  
7  ).asInstanceOf[Expr[Int]]
```

```
1  import quotes.reflect.*  
2  
3  addTerm(  
4    left.asTerm,  
5    right.asTerm  
6  ).asExprOf[Int]
```

Both routes begin from typed `Expr` values. They expose different authoring and implementation surfaces.

Level 2A: `scala.quoted.runtime.impl.ExprImpl`

```
1 def addExprImpl(  
2   left: impl.ExprImpl,  
3   right: impl.ExprImpl  
4 )(using Contexts.Context): impl.ExprImpl =  
5   impl.ExprImpl(  
6     addTpdTree(left.tree, right.tree),  
7     left.scope  
8   )
```

`ExprImpl` exposes the typed Dotty tree and the quote scope carried by the public `Expr`.

This is an internal implementation path, not the public staging abstraction.

It leads to Level 3 `tpd.Tree` while preserving the original scope in the reconstructed `ExprImpl`.

Level 2B: manual reflection and typed quasiquotes

Manual construction

```
1 def addTerm(using Quotes)(  
2   left: quotes.reflect.Term,  
3   right: quotes.reflect.Term  
4 ): quotes.reflect.Term =  
5   import quotes.reflect.*  
6   Select.overloaded(  
7     left,  
8     "+",  
9     List(),  
10    List(right)  
11  )
```

Typed quasiquote

```
1 def addTerm(using Quotes)(  
2   left: quotes.reflect.Term,  
3   right: quotes.reflect.Term  
4 ): quotes.reflect.Term =  
5   qr"$left + $right"
```

Same typed reflection layer, different authoring ergonomics.

Level 2B can also reveal the typed Dotty tree

```
1 def addTerm(using Quotes)(  
2   left: quotes.reflect.Term,  
3   right: quotes.reflect.Term  
4 ): quotes.reflect.Term =  
5   given Contexts.Context =  
6     quotes.asInstanceOf[impl.QuotesImpl].ctx  
7  
8   addTpdTree(  
9     left.asInstanceOf[tpd.Tree],  
10    right.asInstanceOf[tpd.Tree]  
11  ).asInstanceOf[quotes.reflect.Term]
```

```
quotes.reflect.Term  
  ↓ internal cast  
tpd.Tree  
  ↓ typed construction  
tpd.Tree  
  ↓ public reflection view  
quotes.reflect.Term
```

The public result is still ``quotes.reflect.Term``; this path exposes the internal typed tree to perform the construction.

Level 3: `dotty.tools.dotc.ast.tpd.Tree`

```
1  def addTpdTree(  
2    left: tpd.Tree,  
3    right: tpd.Tree  
4  )(using Contexts.Context): tpd.Tree =  
5    tpd.applyOverloaded(  
6      left,  
7      "+".toTermName,  
8      List(right),  
9      Nil,  
10     Types.WildcardType  
11  )
```

This stays inside the typed Dotty representation.

The compiler can perform overload resolution with its internal type information and context.

Level 3 is below the public reflection façade but still after typing.

From Level 3 toward raw untpd

```
1 def addTpdTree(  
2   left: tpd.Tree,  
3   right: tpd.Tree  
4 )(using Contexts.Context): tpd.Tree =  
5   val untyped = addUntpdTree(  
6     untpd.TypedSplice(left),  
7     untpd.TypedSplice(right)  
8   )  
9  
10  new Typer().typedExpr(untyped)
```

`TypedSplice` carries already typed pieces into an untyped construction.

An explicit `Typer` call then returns to Level 3.

This is part of our representation walk. It is not the ordinary compiler direction shown earlier.

Level 4: `dotty.tools.dotc.ast.untpd.Tree`

```
1  def addUntpdTree(  
2    left: untpd.Tree,  
3    right: untpd.Tree  
4  )(using util.SourceFile): untpd.Tree =  
5    untpd.Apply(  
6      untpd.Select(  
7        left,  
8        "+" .toTermName  
9      ),  
10     List(right)  
11  )
```

Macro-Paradise operates at this level before the ordinary typer.

The representation is exact and compiler-version-sensitive.

The typed `qr"..."` family targets quoted reflection, so it cannot serve directly as the authoring surface for this phase.

Typed quasiquotes and pre-typer insertion

Ordinary Scala 3 macro

Macro-Paradise

```
1  source-like typed quasiquote
2    ↓
3  Quotes / quotes.reflect.*
4    ↓
5  typed expansion
```

```
1  parser
2    ↓
3  untpd.Tree
4    ↓ pre-typer transformation
5  ordinary typer
```

Families: `qr/qq` , `tqr/tqq` , `dqr/dqq` .

A `dqr` result is not directly insertable into the pre-typer phase.

Three representation worlds

Scalameta source AST

Neutral semantic model

Exact Dotty `untpd`

`scala.meta.*`

Project-owned and compiler-free.

Exact pre-typer compiler topology.

Broad source-like syntax, parsing, and construction.

Bounded validated Term, Type, and Definition meanings. It may normalize or forget source distinctions.

Compiler-version-specific raw trees.

Scalameta, the neutral model, and exact `untpd` solve different parts of the authoring and lowering problem.

Current source-like to exact lowering path

Scalameta definition or quasiquote
↓
bounded projection / semantic plan
↓
exact-version lowering bridge
↓
positioned untpd.DefDef / ValDef

Macro-Paradise placement helper
↓
pre-typer insertion
↓
ordinary Dotty typer
↓
typed generated API

Some focused public bridges go directly from Scalameta to exact lowering. The architecture does not claim that every API exposes every intermediate layer.

Responsibilities across the lowering pipeline

Scalameta or source frontend

Source-like authoring

Macro-Paradise

Admission, placement, conflicts, composition, rollback, and phase timing

Neutral model

Compiler-free bounded semantics where used

Dotty typer

Authoritative semantic typing of the final program

Quasiquotes exact backend

Lower the admitted shape to exact compiler trees

This separation remains useful even if public names and exact APIs evolve.

A pre-typer tree carries more than source syntax

- 1 AST shape
- 2 + source provenance / attachment
- 3 + spans / positions
- 4 + ownership discipline
- 5 + exact compiler representation
- 6 + correct compiler phase

The generated-origin lowering path attaches deterministic virtual-source provenance to generated definitions.

Macro-Paradise validates output. It does not silently invent missing provenance for arbitrary handler-supplied trees.

Exact lowering remains valuable even when the source-like Scalameta quasiquote looks trivial.

The Macro-Paradise ecosystem

Plugin timing, source-like authoring, exact lowering, and
downstream evidence

Projects and roles

Macro-Paradise

github.com/DmytroMitin/macroparadise-scala3

Compiler plugin and `paradise3.api.*` handler contract.

Quasiquotes

github.com/DmytroMitin/quasiquotes-scala3

Typed quasiquotes, neutral semantics, and exact lowering bridges.

AUXify-scala3

github.com/DmytroMitin/AUXify-scala3

Independent downstream macro-annotation library.

Allow Experimental

github.com/DmytroMitin/allow-experimental

Lets a supported non-inline method implementation call selected experimental APIs without making callers experimental.

`@allowExperimental` is not a macro annotation.

Quasiquote families and AUXify status

Typed quoted-reflection families

AUXify Scala 3 slices

```
1  qr"..."      term construction
2  case qq"..." term matching
3  tqr / tqq      type construction / matching
4  dqr / dqq      definition construction / matching
5  $...           structural splicing
```

Implemented, bounded: @apply , @aux ,
@instance , @delegated , @self

Designed and characterized: @syntax

Postponed: @poly

Ranked or sequence transports remain explicit where required.

The broader target list remains useful, but current support must not be overstated.

Macro-Paradise 0.1.1: released starter and sources

Fastest released starter

Source checkout options

```
1  sbt new DmytroMitin/macroparadise-scala3.g8
```

The public template exercises the Maven Central
0.1.1 line.

```
1  git clone git@github.com:DmytroMitin/\
2  macroparadise-scala3.git
3
4  git clone https://github.com/DmytroMitin/\
5  macroparadise-scala3.git
6
7  gh repo clone DmytroMitin/\
8  macroparadise-scala3
```

Versions, artifacts, and exact compiler identity

Released

Modules and environment

```
1 "com.github.dmytromitin" %  
2   "macroparadise-scala3-plugin" %  
3   "0.1.1"
```

- `macroparadise-scala3-plugin`
- `macroparadise-scala3-plugin-api`
- Package: `paradise3.api.*`
- Scala: `3.3.8`, `3.8.4`, `3.9.0`
- JDK 25
- sbt 1.12.15

Current source build

```
1 "com.github.dmytromitin" %  
2   "macroparadise-scala3-plugin" %  
3   "0.2.0-SNAPSHOT"
```

`CrossVersion.full` reflects raw Dotty/API coupling, not ordinary nearby-version compatibility.

Why handlers are normally precompiled

1. marker annotation
- ↓
2. executable handler implementation
- ↓
3. ordinary annotated consumer
- ↓
- Macro-Paradise phase before typer

The handler receives exact compiler internals and must already be executable when the consumer reaches the plugin phase.

The three-stage build topology follows from phase timing. It is not accidental sbt complexity.

Different-file same-module support exists only as a bounded experimental lane. General same-file automatic handler discovery is not promised.

Exact class-loader identity

Plugin and handler exchange:

```
compiler plugin
      +
precompiled handler
      ↓
shared parent-first runtime universe
      ↓
one exact compiler/API identity
```

- 1 paradise3.api.* contract classes
- 2 Scala runtime classes
- 3 Dotty Tree and Context classes

These must have the same runtime identities, not merely similar source names.

A second incompatible compiler or API copy in a child classloader makes apparently identical raw types incompatible at runtime.

Possible future direct annotation syntax

```
1  import paradise3.api.{
2    ExpansionInput,
3    ExpansionOutcome
4  }
5  import dotty.tools.dotc.core.Contexts
6
7  class myAnnotation
8    extends SomeFutureSuperParadiseAnnotationExpander:
9
10     override def expand(
11       input: ExpansionInput
12     )(using Contexts.Context): ExpansionOutcome =
13       ...
```

The annotation marker and executable handler could share one class surface.

Surface syntax and compiler-integration depth are separate design decisions.

Inheritance does not by itself require the plugin to participate inside ordinary typer.

Future syntax with a precompiled annotation

```
compile myAnnotation first
```

↓

```
compiled class
```

- annotation marker
- expansion protocol

↓

```
compile consumer
```

```
@myAnnotation class A
```

↓ load compiled annotation/expander

```
expand untpd
```

↓

```
ordinary Dotty typer
```

The class combines today's marker, `@expander` metadata, and precompiled `ExpansionHandler` roles without changing the pre-typer model.

Same module, different file

```
1 // Annotation.scala
2 class myAnnotation extends
3   SomeFutureSuperParadiseAnnotationExpander:
4   def expand(...) = ...
5
6 // User.scala
7 @myAnnotation
8 class A
```

```
identify handler source
  ↓ suspend dependent consumers
compile selected handler
  ↓ load freshly compiled output
resume consumers
  ↓ run pre-typer expansion
```

The staging problem returns, but deeper analyzer integration is not the only possible solution.

Same-file source handlers create a bootstrap cycle

```
1 class myAnnotation extends
2   SomeFutureSuperParadiseAnnotationExpander:
3   def expand(...) = ...
4
5 @myAnnotation
6 class A
```

Possible designs

- Special suspension or sub-compilation
- Source splitting or explicit staging
- Continue rejecting this topology
- Integrate more deeply with naming or typing

```
need myAnnotation compiled
to execute @myAnnotation
  ↑
myAnnotation belongs to the unit
we want to transform before typer
```

Deeper integration becomes more attractive here, but the inheritance syntax itself did not force it.

Recognizing the future superclass

Bounded syntactic rule

Read the raw parent name plus a deliberately small import-aware source slice.

This mirrors the current annotation-identity model and keeps pre-typer expansion independent of ordinary typer.

Full semantic resolution

Resolve arbitrary aliases, imports, inherited relationships, and type aliases before expansion.

That requirement makes namer or typer integration much more relevant.

The decisive questions are whether expansion code is already executable and how much semantic recognition the protocol requires before expansion.

Future syntax design space

Already compiled annotation/expander

Current pre-typer ``StandardPlugin`` model is enough.

Same file or arbitrary source handler

Bootstrap problem; deeper integration becomes attractive.

Same module, different file

Suspension and precompilation can still be enough.

Full semantic resolution before expansion

Namer or typer integration becomes more relevant.

Today's indirect ``@expander`` edge keeps the broad/default dependency graph acyclic: an already compiled marker points to already compiled executable expansion code.

Direct annotation syntax has historical precedents

Native Scala 3

Scala 2 Paradise

```
1  @experimental
2  class myAnnotation extends MacroAnnotation:
3    def transform(using Quotes)(
4      tree: quotes.reflect.Definition,
5      companion:
6        Option[quotes.reflect.Definition]
7    ): List[quotes.reflect.Definition] =
8      import quotes.reflect.*
9      ...
```

```
1  @compileTimeOnly("enable macro paradise")
2  class myAnnotation extends StaticAnnotation {
3    def macroTransform(annottees: Any*): Any =
4      macro MyAnnotationMacro.macroTransformImpl
5  }
6
7  object MyAnnotationMacro {
8    def macroTransformImpl(
9      c: whitebox.Context
10    )(annottees: c.Expr[Any]*): c.Expr[Any] = {
11      import c.universe._
12      ...
13    }
14  }
```

Similar surface intent can sit on top of very different staging and compiler-integration contracts.

Current marker and handler syntax

Marker

```
1 import paradise3.api.expander
2 import scala.annotation.StaticAnnotation
3
4 @expander(
5   "com.example.macros.handlers." +
6   "MyAnnotationHandler"
7 )
8 final class myAnnotation
9   extends StaticAnnotation
```

Handler

```
1 import dotty.tools.dotc.core.Contexts.Context
2 import paradise3.api.{
3   ExpansionHandler,
4   ExpansionInput,
5   ExpansionOutcome
6 }
7
8 final class MyAnnotationHandler
9   extends ExpansionHandler:
10   override def annotationName: String =
11     "com.example.macros.annotations.myAnnotation"
12
13   override def expand(
14     input: ExpansionInput
15   )(using Context): ExpansionOutcome =
16     ...
```

`ParadiseAnnotationExpander` and `StructuredExpansionOutput` belong to historical prototype material. Current syntax uses `ExpansionHandler`, `ExpansionChanges`, and `ExpansionEdit`.

What `@expander` connects

`myAnnotation`

- Scala annotation class
- Extends `StaticAnnotation`
- Appears in user source as `@myAnnotation`
- Carries no executable transformation by itself

`@expander(...)`

- Java annotation type in `paradise3.api`
- Runtime-retained class-file metadata
- Stores a fully qualified handler class name
- Lets the plugin discover precompiled executable expansion code

`@myAnnotation` → compiled marker → `@expander("Handler")` → `discovery/loading` → `ExpansionHandler.expand`

@expander is runtime metadata

```
1  package paradise3.api;
2
3  import java.lang.annotation.ElementType;
4  import java.lang.annotation.Retention;
5  import java.lang.annotation.RetentionPolicy;
6  import java.lang.annotation.Target;
7
8  /**
9   * Metadata for precompiled marker annotations.
10   * The value names the ExpansionHandler class.
11   * This metadata does not execute the handler by itself.
12   */
13  @Retention(RetentionPolicy.RUNTIME)
14  @Target({ElementType.TYPE, ElementType.ANNOTATION_TYPE})
15  public @interface expander {
16      String value();
17  }
```

It is neither Scala 2 `macroTransform` nor Scala 3 `MacroAnnotation.transform`.

Project structure follows executable staging

Macro-Paradise default

Scala 2 classic

Native Scala 3

macro-annotations marker project

macros

macro-handlers executable handler project

core depending on macros

core annotated consumer

Analyzer/macro machinery integrates the transform while the consumer compiles.

MacroAnnotation can use compiler suspension and macro loading for same-module different-file compilation.

Macro-Paradise's bounded same-module lane remains separate from its default external-handler path.

sbt integration

Released same-build and published-module setups, then their manual translations

Same-build marker and handler: project definitions

```
1 // project/build.properties
2 sbt.version=1.12.15
3
4 // project/plugins.sbt
5 addSbtPlugin(
6   "com.github.dmytromitin" %
7   "sbt-macroparadise" % "0.1.1"
8 )
9
10 // build.sbt
11 import macroparadise.sbt.MacroParadiseIntegration
12 import macroparadise.sbt.
13   MacroParadisePrecompiledPlugin.autoImport._
14
15 ThisBuild / scalaVersion := "3.3.8"
16 // or exact 3.8.4 / 3.9.0
```

```
macro-annotations
  \ provided compile edge
    core
    \ tool-only expansion edge
macro-handlers
```

This slide begins the released `0.1.1` setup. Current-handler examples later use a locally published `0.2.0-SNAPSHOT` product.

Same-build marker and handler dependencies

Marker project

```
1 lazy val macroAnnotations =
2   (project in file("macro-annotations"))
3   .settings(
4     libraryDependencies += Seq(
5       ("com.github.dmytromitin" %
6         "macroparadise-scala3-plugin-api" %
7         "0.1.1")
8       .cross(CrossVersion.full)
9     )
10  )
```

Handler project

```
1 lazy val macroHandlers =
2   (project in file("macro-handlers"))
3   .settings(
4     libraryDependencies += Seq(
5       ("com.github.dmytromitin" %
6         "macroparadise-scala3-plugin-api" %
7         "0.1.1")
8       .cross(CrossVersion.full),
9       "org.scala-lang" %%
10        "scala3-compiler" % scalaVersion.value
11     )
12  )
```

Same-build consumer and AutoPlugin topology

```
1 lazy val core = (project in file("core"))
2   .dependsOn(
3     macroAnnotations % "provided->compile"
4   )
5   .settings(
6     MacroParadiseIntegration.precompiledProjects(
7       macroAnnotations,
8       macroHandlers
9     )
10  )
11  .enablePlugins(
12    macroparadise.sbt.
13      MacroParadisePrecompiledPlugin
14  )
15  .settings(
16    macroParadiseCompilerProductVersion :=
17      "0.1.1"
18  )
```

The helper can also receive sequences of marker and handler projects.

It derives:

- compiler plugin dependency;
- marker visibility for source compilation;
- complete handler classpath;
- external artifact identity;
- required compiler options.

Published marker and handler coordinates

Marker-producing build

Handler-producing build

```
1 lazy val macroAnnotations =  
2   (project in file("macro-annotations"))  
3   .settings(  
4     moduleName := "my-macro-annotations",  
5     crossVersion := CrossVersion.full  
6   )
```

```
1 lazy val macroHandlers =  
2   (project in file("macro-handlers"))  
3   .settings(  
4     moduleName := "my-macro-handlers",  
5     crossVersion := CrossVersion.full  
6   )
```

Artifacts can come from `publishLocal`, Maven Central, or another repository.

Published modules in the released AutoPlugin setup

```
1  import macroparadise.sbt.MacroParadisePrecompiledPlugin.autoImport._
2
3  ThisBuild / scalaVersion := "3.3.8" // or exact 3.8.4 / 3.9.0
4
5  lazy val core = (project in file("core"))
6    .enablePlugins(macroparadise.sbt.MacroParadisePrecompiledPlugin)
7    .settings(
8      macroParadiseCompilerProductVersion := "0.1.1",
9      macroParadiseMarkerModules := Seq(
10        ("com.example" % "my-macro-annotations" % "1.0.0")
11          .cross(CrossVersion.full)) % Provided
12    ),
13    macroParadiseHandlerModules := Seq(
14      ("com.example" % "my-macro-handlers" % "1.0.0")
15        .cross(CrossVersion.full)
16    )
17  )
```

Marker modules belong to ordinary compile visibility. Handler modules belong to a tool-only loading path.

Current-main 0.2.0-SNAPSHOT sbt boundary

Public released route

- `sbt-macroparadise` 0.1.1
- Maven Central compiler/API product 0.1.1
- Public Giter8 starter
- Released handler protocol

Current source route

- Publish current `sbt-integration` locally
- Publish matching current compiler plugin/API locally
- Select one exact Scala compiler line
- Use current `ExpansionHandler` and `ExpansionEdit` examples

`0.2.0-SNAPSHOT` is a local source-build line, not a remotely published snapshot.

Manual same-build wiring: dependencies

```
1 lazy val core = (project in file("core"))
2   .dependsOn(macroAnnotations)
3   .settings(
4     libraryDependencies +=
5       compilerPlugin(macroparadisePlugin),
6
7     Compile / scalacOptions += {
8       val markerJar =
9         (macroAnnotations / Compile / packageBin).value
10      val handlerJar =
11        (macroHandlers / Compile / packageBin).value
12      val handlerClasses =
13        (macroHandlers / Compile / classDirectory)
14        .value.getCanonicalFile
```

The lower-level translation exposes what the AutoPlugin hides.

The compiler plugin is a compiler dependency. The marker is visible during ordinary compilation. The handler and its runtime closure become a separate executable tool classpath.

Manual same-build wiring: complete handler closure

```
1  val handlerClasspath = handlerJar +:
2    (macroHandlers / Runtime / dependencyClasspath)
3    .value.files
4    .filterNot(
5      _.getCanonicalFile == handlerClasses
6    )
7
8  val buildIdentity =
9    ExternalArtifactIdentity.combined(
10      Seq("marker" -> markerJar),
11      handlerClasspath.zipWithIndex.map {
12        case (file, index) =>
13          f"handler-$index%04d" -> file
14      }
15    )
```

The handler JAR appears first, followed by its effective runtime dependencies.

The identity covers both the marker artifact and the ordered handler classpath so stale or mismatched products do not silently survive incremental compilation.

Manual same-build wiring: compiler inputs

```
1 Seq(  
2   "-Xplugin-require:macroparadise",  
3   s"-P:macroparadise:handlerClasspath=${  
4     handlerClasspath  
5       .map(_.getAbsolutePath)  
6       .mkString(java.io.File.pathSeparator)  
7   }",  
8   s"-P:macroparadise:" +  
9     s"externalArtifactIdentity=sha256:" +  
10    s"$buildIdentity"  
11 )
```

Three essential inputs

1. Require the plugin by name.
2. Supply the complete executable handler path.
3. Supply a content-derived external artifact identity.

The AutoPlugin produces this same semantic contract.

Manual published-module wiring: configuration

```
1  import java.io.File
2  import scala.collection.mutable
3
4  ThisBuild / scalaVersion := "3.9.0"
5
6  val mpVersion = "0.2.0-SNAPSHOT"
7  val producerVersion = "1.0.0"
8  val mpPlugin =
9    ("com.github.dmytromitin" %
10     "macroparadise-scala3-plugin" % mpVersion)
11     .cross(CrossVersion.full)
12
13  val MacroParadiseHandler =
14    config("macroParadiseHandler").hide
```

```
1
2  lazy val markerArtifacts =
3    taskKey[Seq[(String, File)]](
4      "Exact published marker inventory"
5    )
6  lazy val handlerClasspath =
7    taskKey[Seq[(String, File)]](
8      "Complete published handler closure"
9    )
10  lazy val externalArtifactIdentity =
11    taskKey[String](
12      "Combined marker and handler identity"
13    )
```

Current-protocol no-AutoPlugin recipe: copy the public `ExternalArtifactIdentity.scala` helper into `project/`; the hidden configuration keeps executable handler dependencies off the application path.

Manual published-module wiring: requested modules

Requested coordinates

```
1  val markerModules = Seq(  
2    ("com.example" %  
3      "my-macro-annotations" %  
4      producerVersion)  
5    .cross(CrossVersion.full)) %  
6    Provided  
7  )  
8  
9  val handlerModules = Seq(  
10    ("com.example" %  
11      "my-macro-handlers" %  
12      producerVersion)  
13    .cross(CrossVersion.full)  
14  )
```

Exact resolution

```
1  def resolveConfigured(  
2    modules: Seq[ModuleID],  
3    classpath: Classpath,  
4    role: String  
5  ): Seq[(String, File)] =  
6    modules.flatMap { requested =>  
7      val matches = classpath.filter(  
8        _.get(moduleID.key).exists { actual =>  
9          actual.organization == requested.organization &&  
10             (actual.name == requested.name ||  
11               actual.name.startsWith(requested.name + "_")) &&  
12             actual.revision == requested.revision  
13        }  
14      )  
15      require(matches.nonEmpty,  
16        s"configured $role module did not resolve: $requested")  
17      matches.zipWithIndex.map { (entry, index) =>  
18        s"${requested.organization}:${requested.name}: " +  
19        s"${requested.revision}:$index" -> entry.data  
20      }  
21    }
```

Manual published-module wiring: handler closure

```
1  def completeHandlers(  
2    modules: Seq[ModuleID],  
3    classpath: Classpath  
4  ): Seq[(String, File)] = {  
5    val direct = resolveConfigured(  
6      modules, classpath, "handler"  
7    )  
8    val directPaths =  
9      direct.map(_._2.getCanonicalFile).toSet  
10  
11    val transitive = classpath.iterator  
12      .filterNot(entry =>  
13        directPaths(entry.data.getCanonicalFile)  
14      )  
15      .zipWithIndex  
16      .map { (entry, index) =>  
17        val coordinate = entry.get(moduleID.key)  
18          .map(m => s"${m.organization}:${m.name}:${m.revision}")  
19          .getOrElse(entry.data.getName)  
20        f"transitive-${index%04d}:${coordinate}" -> entry.data  
21      }  
22    .toVector
```

Direct handlers come first.

The remaining runtime classpath becomes the transitive closure with stable coordinate labels.

This order matters both for loading and for the identity digest.

Manual published-module wiring: de-duplicate and isolate

Canonical de-duplication

```
1  val seen =  
2    mutable.LinkedHashSet.empty[File]  
3  
4  (direct ++ transitive).filter {  
5    case (_, file) =>  
6      seen.add(file.getCanonicalFile)  
7  }
```

One shared transitive dependency appears exactly once while direct-first order survives.

Consumer configurations

```
1  lazy val core = (project in file("core"))  
2    .configs(MacroParadiseHandler)  
3    .settings(  
4      inConfig(MacroParadiseHandler)(  
5        Defaults.configSettings  
6      )  
7    )  
8    .settings(  
9      libraryDependencies +=  
10        compilerPlugin(mpPlugin),  
11      libraryDependencies += markerModules,  
12      libraryDependencies +=  
13        handlerModules.map(  
14          _ % MacroParadiseHandler.name  
15        )  
16    )
```

Manual published-module wiring: inventories and options

```
1  markerArtifacts := resolveConfigured(
2    markerModules,
3    (Compile / dependencyClasspath).value,
4    "marker"
5  ),
6  handlerClasspath := completeHandlers(
7    handlerModules,
8    (MacroParadiseHandler / dependencyClasspath).value
9  ),
10 externalArtifactIdentity := ExternalArtifactIdentity.combined(
11   markerArtifacts.value,
12   handlerClasspath.value
13 ),
14 Compile / scalacOptions += Seq(
15   "-Xplugin-require:macroparadise",
16   "-P:macroparadise:handlerClasspath=" +
17     handlerClasspath.value.map(_._2.getAbsolutePath)
18       .mkString(File.pathSeparator),
19   "-P:macroparadise:externalArtifactIdentity=sha256:" +
20     externalArtifactIdentity.value
21 )
```

The task graph resolves exact marker artifacts, an ordered executable handler closure, and one combined content identity.

What the complete integration guarantees

- Marker modules are ordinary compile dependencies.
- `% Provided` keeps marker-only artifacts off the application runtime path.
- Handler modules live in a hidden tool-only configuration.
- Direct handlers precede their transitive closure.
- Canonical de-duplication retains each runtime file once.
- The identity covers exact markers and the effective ordered handler path.
- The build emits the plugin requirement, handler path, and SHA-256 identity.
- The larger fixture verifies ordering, isolation, identity, and compiler options.

AutoPlugin settings $\Leftrightarrow$ explicit dependency graph + tool classpath + content identity + compiler options

The handler contract

Plugin-owned transactions and one bounded transformation per handler

Plugin ownership and handler ownership

Plugin owns

- Annotation matching and target admission
- Handler discovery and loading
- Current primary/companion discovery
- Deterministic scheduling
- Package and container conflict checks
- Output validation
- Atomic application and rollback
- Final ordering before typer

Handler owns

One bounded transformation.

It receives a plugin-minted read-only `ExpansionInput` and returns an `ExpansionOutcome`.

The normal contract does not let a handler mutate arbitrary compiler state.

Current target model

Supported public target kinds

- 1 Class
- 2 Trait
- 3 Object

Current execution applies to the established package-level slice. The handler still decides whether its annotation supports the admitted shape.

Not general targets today

- 1 nested / inner / local definitions
- 2 enum / enum case
- 3 method
- 4 val / var
- 5 type alias
- 6 parameter / type parameter
- 7 given
- 8 extension-related forms

Primary, companion, and siblings are revision-local

`ExpansionInput.primary` addresses the currently selected annotated definition.

```
apply requested changes privately
  ↓ discard stale relationship labels
recompute actual companions
  ↓ validate conflicts and topology
rescan current staged program
```

`ExpansionInput.companion` is the actual compatible same-name companion in the current staged scope and unit. Adjacency is not required.

Sibling labels also address the current invocation revision.

Replacements need not preserve the identity of the original primary or companion.

Structured sparse changes

Domain	Supported intentions
primary	preserve, merge, replace, delete
companion	preserve, merge, replace, create, delete
siblings	sparse create, addressed merge, replace, delete

The handler describes changes against the current invocation revision.

The plugin applies them privately, recomputes real relationships, validates the result, and only then continues.

Immutable composable editing

```
1  val edited = for
2    edit0 <- ExpansionEdit.start(input)
3    edit1 <- ExpansionHelpers
4      .placeMembersInPrimary(
5        edit0,
6        List(method.tree, method1.tree, method2.tree)
7      )
8    edit2 <- ExpansionHelpers
9      .placeMembersInCompanion(
10       edit1,
11       List(method3.tree, method4.tree)
12     )
13  yield edit2
14
15  ExpansionEdit.finish(edited)
```

`ExpansionEdit.start(input)` returns:

```
1  Either[ExpansionDiagnostic,
      ExpansionEdit]
```

Each helper threads a new immutable edit through the comprehension. `finish` converts the final `Either` into an `ExpansionOutcome`.

Raw exact replacement

```
1 ExpansionOutcome.Expanded(trees)
```

The expert escape hatch replaces the handler-owned primary/companion region with zero or more raw supported definitions.

```
1 ExpansionOutcome.Expanded(Nil)
```

deletes that owned region.

- Output order is exact.
- No first tree is privileged as the new primary.
- Final topology and conflicts are recomputed.
- The plugin validates before committing.
- Structured helpers remain safer for normal composition.

The handler progression

```
marker / handler architecture
↓
minimal handler: @identity
↓
generated member: @gen
↓
source-like authoring + exact lowering: @addFoo
↓
explicit missing-companion creation
↓
ordinary typer sees A.foo
```

Each example adds one responsibility while retaining the same precompiled, pre-typer protocol.

@identity: current composable form

```
1  import paradise3.api.expander
2  import scala.annotation.StaticAnnotation
3
4  @expander(
5    "com.example.macros.handlers." +
6    "IdentityHandler"
7  )
8  class identity extends StaticAnnotation
```

```
1  final class IdentityHandler
2    extends ExpansionHandler:
3    override def annotationName: String =
4      "com.example.macros.annotations.identity"
5
6    override def expand(
7      input: ExpansionInput
8    )(using Context): ExpansionOutcome =
9      ExpansionEdit.finish(
10        ExpansionEdit.start(input)
11      )
```

The smallest handler proves discovery, binding, invocation, and unchanged pass-through.

@identity: historical prototype APIs

Historical raw form

```
1 final class IdentityHandler
2   extends ParadiseAnnotationExpander:
3   override def annotationName: String =
4     "com.example.macros.annotations.identity"
5
6   override def expand(
7     input: ExpansionInput
8   )(using Contexts.Context): ExpansionOutcome =
9     ExpansionOutcome.Expanded(
10       List(input.annotatedClass)
11     )
```

Historical structured form

```
1 final class IdentityHandler
2   extends ParadiseAnnotationExpander:
3   override def expand(
4     input: ExpansionInput
5   )(using Contexts.Context): ExpansionOutcome =
6     ExpansionOutcome.Structured(
7       StructuredExpansionOutput(
8         primary = input.annotatedClass,
9         companion = input.existingCompanion,
10        additionalTopLevelDefinitions = List()
11      )
12    )
```

Historical prototype: these names show API evolution and must not be mixed into current handler examples.

Historical ExpansionOutcome sketch

```
1 // Historical prototype
2 enum ExpansionOutcome:
3   case Expanded(
4     trees: List[untpd.Tree]
5   )
6   case Structured(
7     output: StructuredExpansionOutput
8   )
9   case Rejected(
10    diagnostics: List[ExpansionDiagnostic],
11    fallback: untpd.TypeDef
12  )
13  case NotApplicable
```

The early design explored both raw list replacement and one structured aggregate.

Current `main` keeps raw exact replacement but models safe composition through current `ExpansionEdit` and `ExpansionChanges` concepts.

@gen : generated instance member

Marker and consumer

Historical prototype helper

```
1  @expander(  
2    "com.example.macros.handlers.GenHandler"  
3  )  
4  final class gen extends StaticAnnotation  
5  
6  @gen  
7  class World  
8  
9  val greeting: String =  
10    new World().generatedHello
```

```
1  final class GenHandler  
2    extends ParadiseAnnotationExpander:  
3    override def annotationName: String =  
4      "com.example.macros.annotations.gen"  
5  
6    override def expand(  
7      input: ExpansionInput  
8    )(using Context): ExpansionOutcome =  
9      ExpansionHelpers.addStringMethodToClass(  
10        input,  
11        methodName = "generatedHello",  
12        value = s"Hello, ${input.className}!"  
13      )
```

@gen : current protocol meaning

```
1  final class GenHandler extends ExpansionHandler:
2    def annotationName =
3      "com.example.macros.annotations.gen"
4
5    def expand(input: ExpansionInput)(using Context) =
6      val method = DefDef(
7        termName("generatedHello"),
8        Nil,
9        Ident(typeName("String")),
10       Literal(Constant(
11         s"Hello, ${input.primary.name}!"
12       ))
13     )
14
15     ExpansionEdit.finish:
16       ExpansionEdit.start(input).flatMap: edit =>
17         ExpansionHelpers.placeMemberInPrimary(
18           edit, method
19         )
```

The canonical current examples use `ExpansionHandler` and `ExpansionEdit`, often with source-like definition lowering.

The semantic point survives the API evolution: a method generated before typer is visible to ordinary source on `World`.

The next example moves the generated API into a companion and authors it through Scalameta.

Current-stage scheduling

```
select next handled annotation
  ↓ expand against current staged tree
validate
  ↓ rescan current trees deterministically
select next current annotation
```

- A later annotation runs only if it still exists.
- Generated handled annotations can become new work.
- Deleting a definition deletes its pending work.
- Stacked annotations see the latest revision.
- The original annotation list is not an immutable queue.

Selection follows source order, tree preorder, and annotation order.

This behavior matters for AUXify composition.

Transactional expansion and rollback

```
annotation 1 succeeds  
annotation 2 succeeds  
annotation 3 fails  
  ↓  
rollback the whole compilation unit
```

Failure classes

- Explicit handler rejection
- Invalid output
- Collision or topology error
- Handler exception
- Stale or duplicate address
- Expansion budget exhaustion

Successful stages remain provisional until the staged result remains valid.

Default budget: **256 successful stages per compilation unit**. It is a termination guard, not a semantic proof.

@addFoo

Scalameta authoring, exact generated-origin lowering, explicit companion creation, and ordinary typing

@addFoo : handler dependencies and marker

Handler project

```
1 lazy val macroHandlers =
2   (project in file("macro-handlers"))
3     .settings(
4       libraryDependencies ++= Seq(
5         ("com.github.dmytromitin" %
6          "macroparadise-scala3-plugin-api" %
7          macroparadiseV)
8         .cross(CrossVersion.full),
9         "org.scala-lang" %%
10          "scala3-compiler" % scalaVersion.value,
11         "org.scalameta" %%
12          "scalameta" % "4.17.3",
13         ("com.github.dmytromitin" %
14          "quasiquotes-scala3-dotty-internal" %
15          quasiquotesV)
16         .cross(CrossVersion.full)
17       )
18     )
```

Marker

```
1 import paradise3.api.expander
2 import scala.annotation.StaticAnnotation
3
4 @expander(
5   "com.example.macros.handlers." +
6   "AddFooHandler"
7 )
8 final class addFoo
9   extends StaticAnnotation
```

The handler combines Macro-Paradise, exact Dotty internals, Scalameta, and the Quasiquotes lowering bridge.

@addFoo : handler entry and target admission

```
1  final class AddFooHandler
2    extends ExpansionHandler:
3
4    override def annotationName: String =
5      "com.example.macros.annotations.addFoo"
6
7    override def expand(
8      input: ExpansionInput
9    )(using Context): ExpansionOutcome =
10      ExpansionEdit.finish:
11        for
12          edit <- ExpansionEdit.start(input)
13          _ <- input.primary match
14            case ExpansionTarget.Class(_) =>
15              Right(())
16            case _ =>
17              Left(ExpansionDiagnostic(
18                "@addFoo requires a class primary",
19                input.currentAnnotation.sourcePos
20              ))
21          // author, lower, place
22        yield result
```

The plugin has admitted a package-level Class/Trait/Object target.

The handler then applies its narrower semantic rule: `@addFoo` requires a class primary.

Failures become positioned `ExpansionDiagnostic`s inside the same immutable edit transaction.

@addFoo : source-like authoring and exact lowering

```
1  definition =
2    q"def foo(x: Int): String = x.toString"
3      .asInstanceOf[Defn.Def]
4
5  lowered <-
6    ScalametaDefinitionGeneratedOriginBridge
7      .lower(
8        definition,
9        "<macroparadise-generated:" +
10         "AddFooHandler:foo>"
11      )
12  .left.map(error =>
13    ExpansionDiagnostic(
14      s"${error.code}: ${error.detail}",
15      input.currentAnnotation.sourcePos
16    )
17  )
```

The `q"..."` here is a **Scalameta quasiquote**, not the typed Quasiquotes `qr"..."` family.

The bridge lowers the admitted source definition to a positioned exact `untpd.DefDef` with deterministic generated-source provenance.

Lowering failures remain explicit and positioned.

@addFoo : explicit missing-companion creation

```
1  result <-  
2    ExpansionHelpers.placeMemberInCompanion(  
3      edit,  
4      lowered.tree,  
5      MissingCompanionPolicy.Create(  
6          ExpansionTargetKind.Object,  
7          DefinitionPlacement.AfterPrimary  
8      )  
9  )  
10  
11  yield result
```

For `@addFoo class A`, there is initially no `object A`.

The helper's default missing-companion policy is `Reject`, so the handler must opt into object creation and specify its placement after the primary.

Creation policy therefore belongs to the explicit handler contract rather than hidden plugin behavior.

@addFoo : the complete current chain

```
1  @addFoo
2  class A
3
4  A.foo(10)
```

```
Scalameta q"def foo ..."
  ↓ generated-origin exact lowering
positioned untpd.DefDef
  ↓ ExpansionEdit
companion Create + placement
  ↓ transactional insertion
ordinary typer sees A.foo
```

The handler authors one source-like method and asks for a missing companion to be created.

This is the central experiment behind rebuilding Macro Paradise.

Historical prototype: primary-member @addFoo

```
1  @expander(  
2    "com.example.macros.handlers.AddFooHandler"  
3  )  
4  final class addFoo extends StaticAnnotation
```

```
1  class AddFooHandler  
2    extends ParadiseAnnotationExpander:  
3    override def annotationName: String =  
4      "com.example.macros.annotations.addFoo"  
5  
6    override def expand(  
7      input: ExpansionInput  
8    )(using Contexts.Context): ExpansionOutcome =  
9      // lower three primary members
```

The marker shape survived into the current design.

ParadiseAnnotationExpander and the older input views identify this block as historical.

Historical prototype: lower three methods

```
1  (
2    ScalametaDefinitionGeneratedOriginBridge.lower(
3      q"def foo(x: Int): String = ${Term.Name(input.annotatedClass.name.show)}.m(x)",
4      "fooFile.scala"
5    ),
6    ScalametaDefinitionGeneratedOriginBridge.lower(
7      q"def bar(x: Int): String = x.toString",
8      "barFile.scala"
9    ),
10   ScalametaDefinitionGeneratedOriginBridge.lower(
11     q"def baz(x: Int): String = bar(x)",
12     "bazFile.scala"
13   )
14 ) match
15   case (Right(method), Right(method1), Right(method2)) =>
16     ExpansionHelpers.placeMembersInPrimary(
17       input,
18       List(method.tree, method1.tree, method2.tree)
19     )
```

Historical prototype: three independently lowered definitions were placed into the annotated class.

Historical prototype: generated primary API

Source

Ordinary calls after expansion

```
1  A().foo(10)
2  A().bar(10)
3  A().baz(10)
```

```
1  @addFoo
2  class A
3
4  object A:
5    def m(x: Int): String =
6      x.toString
```

The example remains useful as API history. Current slides use current handler names and the companion-oriented central example.

AUXify-scala3

A downstream library that tests source decoding, exact lowering, placement, and composition

AUXify exercises the complete architecture

```
source annotation  
↓  
Macro-Paradise scheduling / placement  
↓  
handler source-shape decoding  
↓  
source-like Scalameta construction
```

```
Quasiquotes exact lowering  
↓  
positioned untpd definitions  
↓  
Macro-Paradise placement  
↓  
ordinary Scala typer
```

AUXify asks whether nontrivial user-facing macro-annotation libraries can be built on the pre-typer mechanism.

@apply: type-class materialization

Simple result

```
1  @apply
2  trait Show[A]:
3    def show(a: A): String
4
5  object Show:
6    given Show[String] with
7      def show(a: String): String = a
8
9    // generated:
10    def apply[A](using inst: Show[A]): Show[A] =
11      inst
12
13  val stringShow: Show[String] = Show[String]
```

Path-dependent result

```
1  @apply
2  trait Add[N <: Nat, M <: Nat]:
3    type Out <: Nat
4    def apply(n: N, m: M): Out
5
6  object Add:
7    // generated:
8    def apply[N <: Nat, M <: Nat](
9      using inst: Add[N, M]
10    ): Add[N, M] { type Out = inst.Out } =
11      inst
```

The result preserves the instance's dependent `Out` member.

@apply: marker and historical handler sketch

```
1  @expander(  
2    "com.github.dmytromitin.auxify." +  
3    "macros.internal.ApplyHandler"  
4  )  
5  class apply extends StaticAnnotation
```

```
1  // Historical handler surface retained from v8  
2  final class ApplyHandler  
3    extends ParadiseAnnotationExpander:  
4    override val annotationName: String =  
5      "com.github.dmytromitin.auxify.macros.apply"  
6  
7    override def expand(  
8      input: ExpansionInput  
9    )(using Context): ExpansionOutcome =  
10      ExpansionHelpers.withAnnotatedClassView(input):  
view =>  
11        view.typeParameters match  
12          case List(typeParameter) =>  
13            lowerAndPlace(  
14              input,  
15              ApplyDefinitionBuilder.lower(  
16                input.className,  
17                typeParameter.name  
18              )  
19            )
```

The public marker names the precompiled handler.

The code shows the admitted source shape and lowering flow. The handler base name is historical and is not presented as the current Macro-Paradise API.

@apply: source-like definition and exact placement

Author and lower

Place the exact tree

```
1 private[internal] object ApplyDefinitionBuilder:
2   def definition(
3     className: String,
4     typeParameterName: String
5   ): Defn.Def =
6     val cls = Type.Name(className)
7     val param = Type.Name(typeParameterName)
8     val tparamTree = tparam"$param"
9     val target = t"$cls[$param]"
10    q"def apply[$tparamTree](using inst: $target):
11      $target = inst"
12
13  def lower(...)(using Context) =
14    ContextualMethodPeerBridge.lower(
15      definition(className, typeParameterName),
16      s"... "
17    )
```

```
1 private def lowerAndPlace(
2   input: ExpansionInput,
3   lowered: Either[
4     ContextualMethodPeerBridge.Failure,
5     ContextualMethodPeerBridge.Lowered
6   ]
7 )(using Context): ExpansionOutcome =
8   lowered match
9     case Right(value) =>
10     ExpansionHelpers.addMethodToCompanion(
11       input,
12       value.tree,
13       CompanionMethodConflictPolicy
14         .PreserveExisting
15     )
```

Scalameta authors the definition, Quasiquotes lowers it, and Macro-Paradise places it.

@aux : type-level programming helper

Source

Generated companion member

```
1  @aux
2  trait Add[N <: Nat, M <: Nat]:
3    type Out <: Nat
4    def apply(n: N, m: M): Out
```

```
1  object Add:
2    type Aux[
3      N <: Nat,
4      M <: Nat,
5      Out0 <: Nat
6    ] = Add[N, M] {
7      type Out = Out0
8    }
```

```
1  @expander(
2    "com.github.dmytromitin.auxify." +
3    "macros.internal.AuxHandler"
4  )
5  class aux extends StaticAnnotation
```

The alias names the type class together with an exact result-type refinement.

@aux : decode the supported source shape

```
1 // Historical handler surface retained from v8
2 final class AuxHandler
3     extends ParadiseAnnotationExpander:
4     override val annotationName: String =
5         "com.github.dmytromitin.auxify.macros.aux"
6
7     override def expand(
8         input: ExpansionInput
9     )(using Context): ExpansionOutcome =
10         AuxHandler.expandWithLowering(input):
11             (shape, context) =>
12                 AuxDefinitionBuilder.lower(shape)(using
13                     context)
```

```
1
2 private[internal] object AuxHandler:
3     def expandWithLowering(input: ExpansionInput)(
4         lower: Lowering
5     )(using Context): ExpansionOutcome =
6         input.annotatedClassTypeStructureView match
7             case Right(structure) =>
8                 AuxSourceShapeDecoder
9                     .decode(input.className, structure) match
10                     case Right(shape) =>
11                         lower(shape, summon[Context]) match
12                             case Right(lowered) =>
13
14 ExpansionHelpers.addTypeToCompanion(
15     input,
16     lowered.tree,
17     CompanionTypeConflictPolicy.PreserveExisting
18 )
```

annotated raw class view → bounded structural view → validated semantic shape → exact lowering → companion type placement

Every arrow admits only a bounded source family and reports failures rather than guessing.

@aux : author the alias parameters and target

```
1 private[internal] object AuxDefinitionBuilder:
2   def definition(
3     shape: AuxSourceShapeDecoder.Shape
4   ): Defn.Type =
5     val aliasName = Type.Name("Aux")
6     val typeClassName =
7       Type.Name(shape.typeClassName)
8     val first =
9       Type.Name(shape.firstTypeParameterName)
10    val second =
11      Type.Name(shape.secondTypeParameterName)
12    val upper = Type.Name(shape.upperBoundTypeName)
13    val resultMember =
14      Type.Name(shape.resultTypeMemberName)
15    val output =
16      Type.Name(shape.generatedResultParameterName)
```

```
1
2   val firstParam: Type.Param =
3     tparam"$first <: $upper"
4   val secondParam: Type.Param =
5     tparam"$second <: $upper"
6   val outputParam: Type.Param =
7     tparam"$output <: $upper"
8
9   val allParams =
10     List(firstParam, secondParam, outputParam)
11   val target =
12     t"$typeClassName[..${List(first, second)}]"
13
14   val resultEquality: Defn.Type =
15     q"type $resultMember = $output"
16
17   val refinement: Type =
18     t"$target { ..${List(resultEquality)} }"
19
20   q"type $aliasName[..$allParams] = $refinement"
```

The decoder recovers the type-class name, two inputs, shared bound, result member, and output parameter; the builder renders those exact roles as Scalameta trees without consulting Dotty symbols.

@aux: refinement and exact lowering

```
1  def lower(shape: Shape)(using Context) =  
2    AuxTypeAliasPeerBridge.lower(  
3      definition(shape),  
4      expectedAliasName = "Aux",  
5      expectedFirstParameterName =  
6        shape.firstTypeParameterName,  
7      expectedFirstUpperBoundName =  
8        shape.upperBoundTypeName,  
9      expectedSecondParameterName =  
10       shape.secondTypeParameterName,  
11      expectedSecondUpperBoundName =  
12       shape.upperBoundTypeName,  
13      expectedOutputParameterName =  
14       shape.generatedResultParameterName,  
15      expectedOutputUpperBoundName =  
16       shape.upperBoundTypeName,  
17      expectedTargetName = shape.typeClassName,  
18      expectedRefinementMemberName =  
19       shape.resultTypeMemberName,  
20      virtualSourceName = s"..."  
21    )
```

The bridge receives both the authored definition and the exact semantic expectations recovered from the source.

It can reject a source-like tree that does not match the intended bounded meaning before producing compiler-specific `untpd` .

Virtual source identity preserves generated-origin provenance.

@instance: construct type-class instances

Source

Generated conceptually

```
1  @instance
2  trait Monoid[A]:
3    def empty: A
4    def combine(a: A, a1: A): A
```

Current AUXify implements a bounded Scala 3 slice for this shape rather than full Scala 2 parity.

```
1  object Monoid:
2    def instance[A](
3      emptyValue: => A,
4      combineFunction: (A, A) => A
5    ): Monoid[A] =
6      new Monoid[A]:
7        override def empty: A =
8          emptyValue
9        override def combine(
10          a: A,
11          a1: A
12        ): A =
13          combineFunction(a, a1)
```

@delegated: generate a type-class forwarder

Source

Generated conceptually

```
1  @delegated
2  trait Show[A]:
3    def show(a: A): String
```

```
1  object Show:
2    def show[A](a: A)(
3      using inst: Show[A]
4    ): String =
5      inst.show(a)
```

A first bounded Scala 3 slice is implemented.

The companion method delegates to the contextual type-class instance.

@syntax: selected Scala 3 design

Source

```
1  @syntax
2  trait Monoid[A]:
3    def empty: A
4    def combine(a: A, a1: A): A
```

Intended generated syntax

```
1  object Monoid:
2    object syntax:
3      extension [A](a: A)
4        def combine(a1: A)(
5          using inst: Monoid[A]
6        ): A =
7          inst.combine(a, a1)
```

The familiar import remains:

```
1  import Monoid.syntax.*
```

Current status: **characterized and designed, not implemented.**

@self: broader parity target

Original source shape

```
1  @self
2  sealed trait Nat:
3    type ++ = Succ[Self]
4
5  @self
6  case object _0 extends Nat
7
8  type _0 = _0.type
9
10 @self
11 case class Succ[N <: Nat](n: N)
12   extends Nat
```

Conceptual expansion

```
1  sealed trait Nat: self =>
2    type Self >: self.type <:
3      Nat { type Self = self.Self }
4    type ++ = Succ[Self]
5
6  case object _0 extends Nat:
7    override type Self = _0
8
9  type _0 = _0.type
10
11 case class Succ[N <: Nat](n: N)
12   extends Nat:
13     override type Self = Succ[N]
```

@self: current implementation boundary

Implemented first slice

- Plain trait
- Zero type parameters
- Default self-type semantics
- Bounded source shape

Not general today

- Classes and objects
- Generic targets
- The full `Nat`, `_0`, and `Succ` parity example
- All Scala 2 AUXify semantics

The previous slide records the broader semantic target. It is not a claim that the complete example currently compiles.

Composition is stronger evidence than isolated demos

```
1  @apply
2  @instance
3  trait ApplyThenInstance[A]:
4    def empty: A
5    def combine(a: A, a1: A): A
```

```
1  @instance
2  @apply
3  trait InstanceThenApply[A]:
4    def empty: A
5    def combine(a: A, a1: A): A
```

Both source orders are qualified for bounded `@apply + @instance`. Bounded `@apply + @aux` and `@apply + @delegated` slices are also qualified in both orders. The second handler must see the current result of the first.

Rewriting existing definitions is a different problem

Intended transformation

```
existing untpd
  ↓ bounded exact capture / view
preserve raw handles for unchanged pieces
  ↓ author and lower changed fragments
validated reconstruction plan
  ↓
exact untpd
```

```
1  @addOption
2  def foo(x: Int): String = rhs
3
4  // becomes
5
6  def foo(x: Int): Option[String] =
7    Option(rhs)
```

Fresh generation is only one direction. Rewriting must preserve unchanged parts of an existing compiler-owned graph.

This must not become a blind round-trip of the whole owner through Scalameta.

Q / N / U-D / U-U / C

One source-like ambition splits into typed staging, neutral semantics, exact lowering, exact rewriting, and composition policy

Five independent architecture questions

1. What is the source-like authoring and pattern language?

2. What semantic representation sits between frontends and backends?

3. Is the target typed quoted reflection or raw pre-typer Dotty?

4. Are we generating a fresh tree or transforming an existing owner?

5. How do layers compose into public APIs and downstream integrations?

Q	typed Quotes-aware frontend
↓	
N	compiler-free semantic model
↓	
U-D	fresh exact untpd lowering
U-U	existing untpd capture/rewrite
C	cross-layer composition policy

Q and N

Q: Quotes-aware typed frontend

- Typed Scala 3 macro world
- `Quotes` , `Expr` , `quotes.reflect`
- Public `qr/qq` , `tqr/tqq` , `dqr/dqq`
- Staging, scope, and typed reflection constraints

N: neutral semantic model

- Compiler-free project representation
- Bounded Term, Type, and Definition meanings
- Scalameta interoperability
- Can normalize or intentionally forget source distinctions

Q and N can share source-like ideas without pretending that typed staging and compiler-free semantics are the same representation.

U-D and U-U

U-D: fresh untyped lowering

Build new exact Dotty `untpd` trees from validated semantic or source plans.

- 1 fresh definition plan
- 2 ↓ exact-version lowering
- 3 new untpd tree

Generated-origin `@addFoo` follows this direction.

U-U: existing-tree transformation

Capture, preserve, rewrite, and reconstruct an existing exact `untpd` graph.

- 1 existing untpd owner
- 2 ↓ preserve exact handles
- 3 reconstructed untpd owner

`@addOption` motivates this separate direction.

C composes layers; it is not another AST universe

C: integration and API policy

Non-equivalences

- Which frontends can feed which backends?
- Which representations are public?
- Where do exact-version boundaries appear?
- How do downstream integrations preserve provenance and errors?
- Which source-like syntax remains honest for each target phase?

```
1  N != public n* syntax
2  U != public u* syntax
3  C != another tree universe
```

Possible future `n*` or `u*` names are surface choices, not definitions of the internal labels.

One source-like syntax is honest only when it lowers differently for the compiler phase and representation where the result will be used.

Scala 2 felt closer to one tree world

Scala 2

Macro APIs and quasiquotes shared a broad reflection/compiler `Tree` universe.

That made source-like quasiquotes natural both for ordinary macros and for Macro-Paradise annotees.

Scala 3

```
1  public typed staged reflection
2    Quotes / Expr / quotes.reflect
3
4  compiler-internal phase trees
5    untpd / tpd / Context
```

Macro-Paradise chooses the internal pre-typer side because source-visible generated definitions must participate in ordinary typing.

Current Macro-Paradise contract

- Scala 3 `StandardPlugin`
- Package-level `Class/Trait/Object` targets
- Exact Scala `3.3.8` , `3.8.4` , `3.9.0`
- Compiler-sensitive `ExpansionHandler` API
- Precompiled external handlers by default
- Bounded same-module experiment
- Deterministic current-tree scheduling
- Transactional validation and rollback
- Structured changes
- Raw exact replacement escape hatch

“Experimental” has concrete target, version, staging, and representation boundaries.

Claims the current work does not support

- Macro-Paradise is a Scala 3 `ResearchPlugin`
- General nested or local targets
- Arbitrary method, val, type, given, or extension targets
- Compiler-version-independent handler binaries
- General same-file source handler discovery
- Arbitrary Scalameta to arbitrary `untpd` lowering
- Typed `dqr` results can be inserted directly pre-typer
- AUXify `@syntax` is implemented
- The broad `@self` parity example is implemented
- A 256-stage budget proves semantic termination

Each negative boundary protects the distinction between demonstrated slices and broader future architecture.

What works today: one complete story

```
1  @addFoo
2  class A
3
4  A.foo(10)
```

1. `@addFoo` is an ordinary marker carrying `@expander` metadata.

2. A `StandardPlugin` phase runs after parser and before typer.

3. The exact-line precompiled handler is loaded.

4. The handler authors `def foo...` with Scalameta.

5. Quasiquotes lowers the supported definition to positioned exact `untpd` .

6. `ExpansionEdit` explicitly creates the missing companion and places the member.

7. Macro-Paradise validates the transaction and rescans the current program.

8. Ordinary Dotty typer sees the generated `A.foo` .

marker → handler → source-like definition → exact
untpd → structured edit → validated program → typer

Native MacroAnnotation and Macro-Paradise optimize different semantics

Native Scala 3

```
typed macro-expansion world
  ↓ strong typed reflection
generated definitions
  ↓ limited external visibility
typed expansion result
```

The typed API constrains how generated definitions participate outside the expansion.

Macro-Paradise experiment

```
parsed untyped program
  ↓ pre-typer transformation
generated definitions
  ↓ ordinary compiler input
normal name resolution and typing
```

The raw contract accepts greater compiler coupling and authoring responsibility.

Neither contract is universally better. They choose different safety, visibility, and compiler-integration trade-offs.

Costs of the pre-typer approach

- Compiler-internal API dependence
- Harder IDE, incremental compiler, and same-module behavior
- Exact Scala version coupling
- Less help from public typed reflection
- Responsibility for valid raw trees
- More work for diagnostics and composition
- Explicit source provenance and positions
- Explicit rollback and validation machinery
- More complicated handler loading and build topology
- Separate exact rewriting architecture for existing owners

The experiment asks whether source-visible generated APIs justify these costs for libraries that genuinely require them.

Possible future directions

Macro-Paradise

- Broader targets
- More same-module and IDE qualification
- More ergonomic handler authoring
- More direct marker/handler syntax
- Transactional transformation APIs

Quasiquotes

- Broader neutral Term/Type/Definition semantics
- Broader exact lowering
- Better compiler-neutral authoring
- Existing-tree capture and rewrite
- Possible future `n*` or `u*` surface names

AUXify

- Widen bounded annotation slices
- Implement selected `@syntax`
- Continue Scala 2 parity where Scala 3 has an appropriate semantic counterpart

Scala Semantic Harness: feedback wanted

github.com/DmytroMitin/scala-semantic-harness

Bring one concrete Scala project decision and compare the harness result with:

Early-feedback guide:

github.com/DmytroMitin/scala-semantic-harness/blob/main/docs/early-feedback.md

- the compiler;
- the project's tests;
- the tooling and evidence you already trust.

Canonical skill:

`skills/semantic-scala/SKILL.md`

The useful feedback is where provenance, freshness, uncertainty, or semantic scope needs to become more explicit.

Repositories and contact

Projects

- github.com/DmytroMitin/macroparadise-talk-09-2026
- github.com/DmytroMitin/macroparadise-scala3
- github.com/DmytroMitin/quasiquotes-scala3
- github.com/DmytroMitin/AUXify-scala3
- github.com/DmytroMitin/allow-experimental
- github.com/DmytroMitin/scala-semantic-harness

Contact

- LinkedIn: [linkedin.com/in/dmitin](https://www.linkedin.com/in/dmitin)
- GitHub: github.com/DmytroMitin
- Facebook: [facebook.com/dmitry.mitin](https://www.facebook.com/dmitry.mitin)
- X: x.com/DmytroMitin
- Instagram: [instagram.com/dmytro.mitin](https://www.instagram.com/dmytro.mitin)
- Threads: [threads.com/@dmytro.mitin](https://www.threads.com/@dmytro.mitin)
- YouTube: [youtube.com/@DmytroMitin](https://www.youtube.com/@DmytroMitin)
- Email: dmitin3@gmail.com